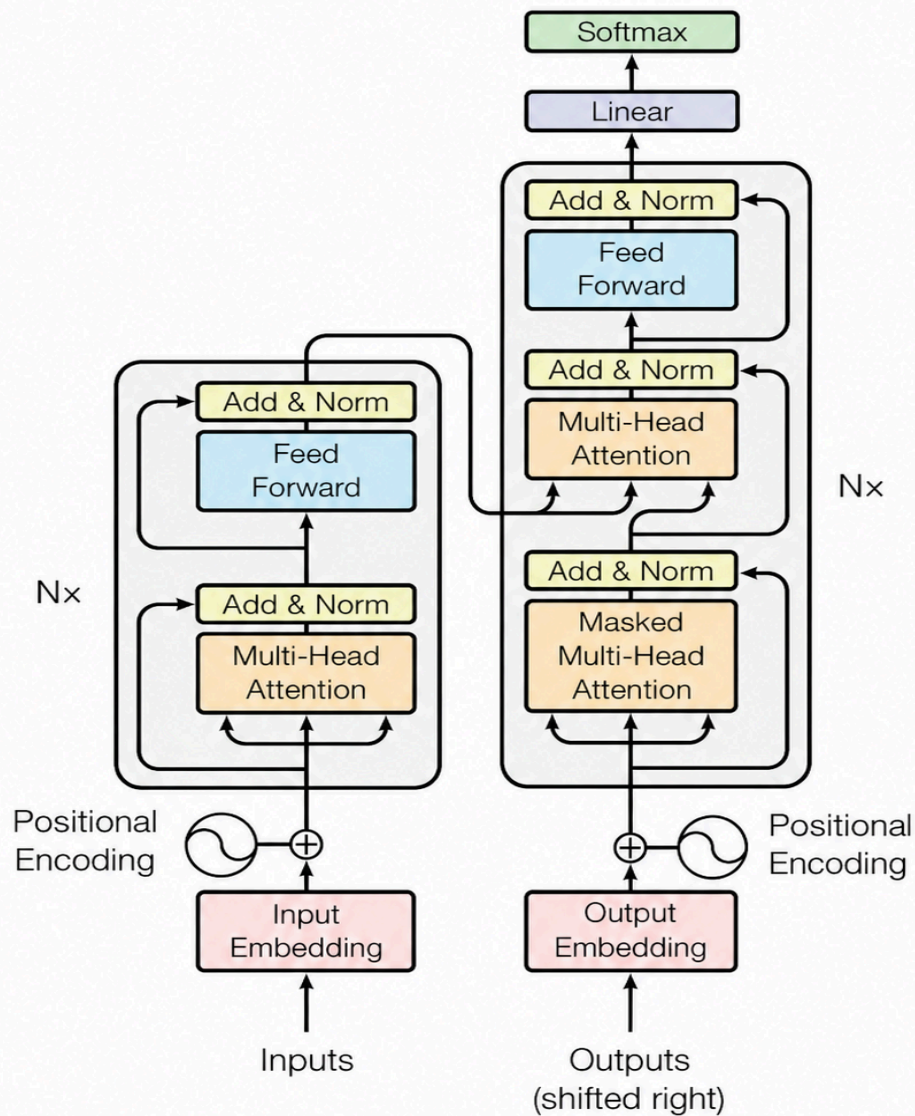




Encoder Side

1. Input Embedding

Converts each input token (word/subword) into a dense vector representation.

These embeddings capture semantic meaning so the model can process language numerically.

2. Positional Encoding

Adds positional information to embeddings because Transformers do not process tokens sequentially like RNNs.

It helps the model understand token order such as first word, second word, etc.

3. Multi-Head Attention

Allows the encoder to focus on different words of the sentence simultaneously.

Multiple attention heads learn different relationships like grammar, context, and semantic dependency.

4. Add & Norm

Performs residual connection followed by layer normalization.

This stabilizes training and helps prevent vanishing gradient problems in deep networks.

5. Feed Forward

A fully connected neural network applied independently to each token.

It helps the model learn more complex feature transformations after attention.

6. Nx

Indicates that the encoder block is repeated N times.

Stacking multiple layers allows deeper contextual understanding.

Decoder Side

7. Output Embedding

Converts previously generated output tokens into vector embeddings.

These embeddings are used by the decoder to predict the next token.

8. Positional Encoding

Adds order information to decoder input tokens.
This ensures the decoder understands token positions during generation.

9. Masked Multi-Head Attention

Prevents the decoder from seeing future words during training.
This ensures autoregressive generation where prediction depends only on past tokens.

10. Add & Norm

Again applies residual connection and normalization after masked attention.
This improves gradient flow and training stability.

11. Multi-Head Attention (Encoder-Decoder Attention)

Allows the decoder to attend to encoder outputs.
This connects input sentence understanding with output generation.

12. Feed Forward

Processes decoder representations through dense neural layers.
It refines token-level features before final prediction.

13. Linear

Projects decoder output vectors into vocabulary-sized logits.
Essentially converts hidden representations into word prediction scores.

14. Softmax

Converts logits into probability distributions over the vocabulary.

The token with highest probability is selected as the next generated word.

Extra Important Concept

15. Outputs (shifted right)

During training, decoder inputs are shifted by one position.

This teaches the model to predict the next token based on previous tokens only.

Residual connection means the model does not only use the transformed output of a layer, it also directly keeps the original input and adds it back later.

Mathematically:

$$\text{Output} = F(x) + x$$

Where:

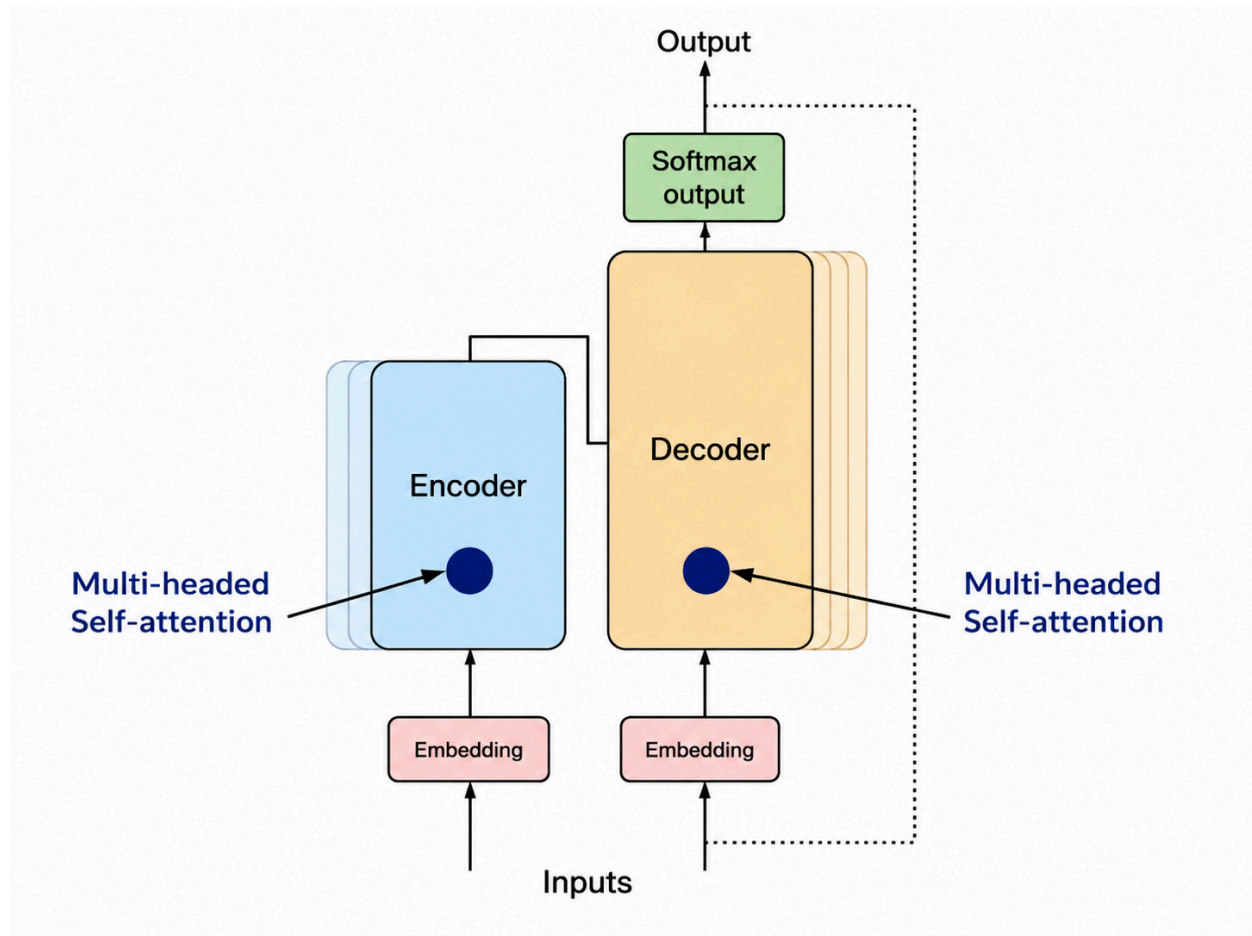
- x = original input
- $F(x)$ = transformed output from a layer (like attention or feed-forward)
- $F(x) + x$ = residual connection result

In Simple Words -

Instead of saying: "Forget the original input, only use the new transformed data"

The Transformer says: "Use the transformed data, but also preserve the original information."

That preserved shortcut path is called the **residual connection** or **skip connection**.



Encoder:

The encoder reads the entire input sentence and converts it into contextual representations. It understands relationships between all words before passing information to the decoder.

Logistics Example -

Suppose TSP email says:

Truck is waiting for unloading at Hamburg warehouse since morning

Encoder understands:

- “truck” relates to logistics object
- “waiting for unloading” indicates scenario intent
- “Hamburg warehouse” is location context

So the encoder builds semantic understanding of the whole email.

Decoder:

The decoder generates the output token by token using encoder information. It predicts the next word while considering previously generated words.

Logistics Example-

After encoder understands the email, decoder may generate:

Scenario = **Unloading**

The decoder creates this response sequentially.

Embedding:

Embedding converts words into dense numerical vectors.

Similar words get similar vector representations in high-dimensional space.

Logistics Example

Words like:

truck, shipment, carrier, transport

will have nearby vector meanings because they belong to logistics context.

Similarly:

delay, waiting, hold, pending

may cluster semantically.

Multi-Head Self-Attention:

Self-attention allows **each word to look at other words in the same sentence.**

Multi-head means multiple attention mechanisms learn different relationships simultaneously.

Logistics Example

Email:

Truck reached warehouse but unloading has not started

One attention head may focus on:

- Truck ↔ warehouse

Another may focus on:

- unloading ↔ not started

Another may focus on:

- temporal relationship

This is why Transformers understand context very well.

Softmax:

Softmax converts raw prediction scores into probabilities.

The highest probability token becomes the output prediction.

Logistics Example

Model logits:

Loading = 2.1

Unloading = 8.7

WH slot = 1.2

After Softmax:

Waiting for Unloading = 97%

Model selects that scenario.

Let's store Pi: 3.141592

3.1415920257568359375

Meaning FP32 can represent very huge positive and negative values.

For FP16, the range becomes much smaller:

-65504 to +65504

because FP16 uses fewer exponent bits.

Type	Memory	Exponent Bits	Precision	Range
FP32	4 bytes	8 bits	High	Very large
FP16	2 bytes	5 bits	Lower	Smaller

The important concept is:

- More exponent bits → larger range of numbers
- More fraction bits → higher precision
- Fewer total bits → lower memory usage

In deep learning quantization:

- FP16 reduces GPU memory usage
- FP16 speeds up training/inference
- But FP16 loses numerical precision compared to FP32

This is why modern GPUs use mixed precision training:

- calculations partly in FP16
- sensitive operations in FP32

> What is the primary goal for quantization in model training?

Reduce memory required for models

> What kind of operation in deep learning does CUDA primarily accelerate?

Matrix multiplication operations

> Which optimizer-related memory components are specifically mentioned as using GPU memory during training?

Two Adam optimizer states

What is Distributed Data Parallel (DDP)?

Distributed Data Parallel (DDP) is a training technique where the same model is copied across multiple GPUs, and each GPU trains on a different portion of the dataset simultaneously.

Instead of:

1 GPU $\rightarrow$ 1000 emails

we do:

GPU 1 $\rightarrow$ 250 emails

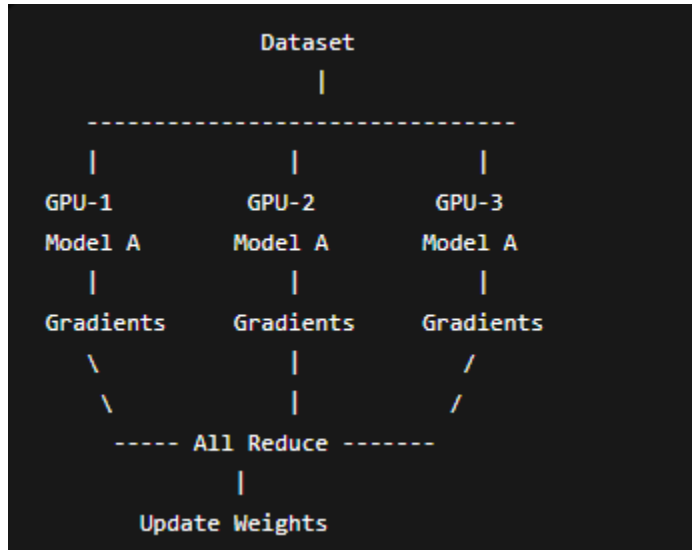
GPU 2 $\rightarrow$ 250 emails

GPU 3 $\rightarrow$ 250 emails

GPU 4 $\rightarrow$ 250 emails

This makes training much faster.

Architecture



Each GPU has:

- Same model
- Different data batch
- Own forward pass
- Own backward pass

Then gradients are synchronized.

10 million shipment emails -

Categories:

- Delivery Delay
- Missing Document
- Route Change
- Invoice Query

Using DDP with 4 GPUs:

GPU 1 → 2.5M emails

GPU 2 → 2.5M emails

GPU 3 → 2.5M emails

GPU 4 → 2.5M emails

Step 1: Model Replication

The same model is copied to all GPUs.

GPU 1 → BERT

GPU 2 → BERT

GPU 3 → BERT

GPU 4 → BERT

All start with identical weights.

Step 2: Different Data

Each GPU receives a different mini-batch.

Delivery delay due to rain

Missing Document, employee absent

Route change due to road block

Step 3: Forward Pass

Each GPU computes predictions independently.

Input Email

↓

Transformer

↓

Prediction

Step 4: Backward Pass

Each GPU computes gradients independently.

GPU 1 → Gradient G1

GPU 2 → Gradient G2

GPU 3 → Gradient G3

GPU 4 → Gradient G4

Step 5: All-Reduce

Now all GPUs share gradients.

Mathematically:

$$G_{avg} = \frac{G_1 + G_2 + G_3 + G_4}{4}$$

Every GPU gets the same averaged gradient.

This operation is called **All-Reduce**.

Step 6: Weight Update

Each GPU updates weights using the same averaged gradient.

Result:

GPU 1 weights = GPU 2 weights
= GPU 3 weights
= GPU 4 weights

Step 6 means -

In DDP, all GPUs have:

Same model architecture

Same initial weights

Same optimizer configuration

Real Intuition

Think of 4 students preparing for an exam.

Initially:

Student 1 = same book
Student 2 = same book
Student 3 = same book
Student 4 = same book



Each studies different chapters.

At the end of the day:

All students share notes



Then everyone updates their book with the combined notes.

Result:

All books become identical again



That is exactly what DDP does with model weights.

Zero Redundancy Optimizer (ZeRO) - Reduces memory by distributing (sharding) the model parameters, gradients, and optimizer states across GPUs.

FSDP -

Problem with DDP

Suppose you have a huge model:
70 Billion parameters

In DDP:

GPU 1 -> Full Model

GPU 2 -> Full Model

GPU 3 -> Full Model

GPU 4 -> Full Model

Every GPU stores:

- Full model weights
- Full gradients
- Full optimizer states

If model size is:

100 GB

then:

GPU 1 = 100 GB

GPU 2 = 100 GB

GPU 3 = 100 GB

GPU 4 = 100 GB

Total:

400 GB

Very wasteful

FSDP = Fully Sharded Data Parallel

Instead of keeping a full model on every GPU, FSDP splits (shards) the model across GPUs.

DDP

GPU 1 -> Full Model

GPU 2 -> Full Model

GPU 3 -> Full Model

GPU 4 -> Full Model

FSDP

GPU 1 -> 25% of Model

GPU 2 -> 25% of Model

GPU 3 -> 25% of Model

GPU 4 -> 25% of Model

Now memory usage drops dramatically.

Example

Suppose model has:

100 million parameters

Split across 4 GPUs.

DDP

GPU 1 -> 100M

GPU 2 -> 100M

GPU 3 -> 100M

GPU 4 -> 100M

Total memory:

400M parameters stored

FSDP

GPU 1 -> 25M

GPU 2 -> 25M

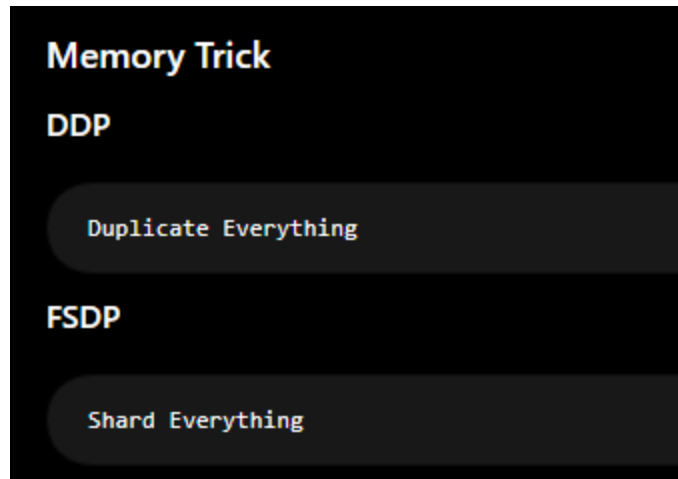
GPU 3 -> 25M

GPU 4 -> 25M

Total:

100M parameters stored

Huge memory saving.



Data parallelism is a strategy that splits the training data across multiple GPUs. Each GPU processes a different subset of the data simultaneously, which can greatly speed up the overall training time.

After an FSDP operation, what happens by default to non-local unshared data?
Release back as shared data

Scaling choices for pre-training -
Goal: maximize model performance ?

SCALING CHOICE: -
1. Dataset size (Number of tokens)

SCALING CHOICE -
2. Model size (Number of parameters)